

Lego Minifigs Classification Using Transfer Learning

*^{1,2}, †², and ‡^{1,3}

1

2

3

May, 2026

Abstract

Identifying and categorizing objects with overlapping visual attributes is a complex challenge in computer vision. This project tackles the multilabel classification of Lego minifigures, a scenario where a single subject frequently belongs to multiple thematic categories simultaneously (e.g., both "Basic" and "LEGOLAND"). To address this, we implement a transfer learning pipeline utilizing a pre-trained ResNet18 convolutional neural network modified with a custom 20-node fully connected head. To combat the severe long-tail class imbalance inherent in the dataset, we employ a dual-stage gradient correction strategy: inverse-frequency oversampling alongside a weighted Binary Cross-Entropy loss function (BCEWithLogitsLoss), further stabilized by geometric data augmentations. The model's performance was evaluated using the F1-score, demonstrating high predictive efficacy for themes with distinct morphological features (such as Friends or Duplo) while highlighting the mathematical difficulty of isolating visually generic themes (such as Castle). Finally, Gradient-weighted Class Activation Mapping (Grad-CAM) was utilized to verify model interpretability, confirming that the network successfully learned to base its predictions on logical, structural features rather than relying on background artifacts or data leakage.

1 Introduction

This project constructs an end-to-end deep learning computer vision pipeline for image classification, using a pre-trained architecture via transfer learning. We build our model on a Lego Minifigure dataset containing 17632 different figures from various set themes.

The original data comes from two different files (see Figure 1). The first one is a JSON file containing all information about one lego minifigure, including its name, release year, category, existing themes and values. The second file contains the image of that figure, in a JPEG format. As described in a JSON file, a minifigure can have a varied number of different themes (e.g., Gears, Star Wars). Our task is to predict the set of themes associated with a given figure. To achieve this, we first encode our

*

†

‡

```

{
  "id": 2740,
  "name": "Truck Driver with Brown Beard...",
  "link": "/minifigs/4j001/truck-driver...",
  "year": 2003,
  "img_url": "https://img.bricklink.com/...",
  "category": "For Juniors",
  "themes": [
    "4 Juniors"
  ]
}

```

(a) JSON Metadata



(b) Visual Representation

Figure 1: Data pipeline inputs. The raw JSON metadata (a) is paired with its corresponding image (b) to form a single training instance.

raw data using MultilabelBinarizer to turn our initial theme list into a binary vector. We assessed at the frequency of theme counts and chose the top 20 themes to ensure that each theme has sufficient representation across all data splits. Furthermore, we employ further techniques to prevent the negative effects of class imbalance, which will be addressed in the coming section.

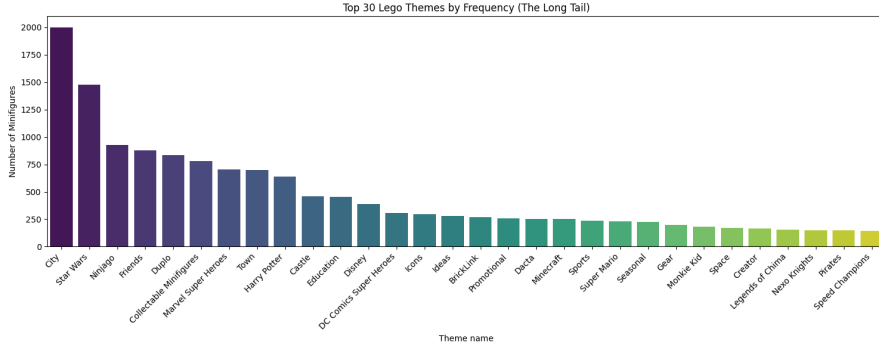


Figure 2: Frequency of top 30 minifigure themes

2 Methodology

2.1 Architecture & Transfer Learning

We utilize ResNet-18 [1], pre-trained on the ImageNet dataset [2], as our choice of base model for transfer learning. A defining characteristic of the ResNet architecture is its use of residual (skip) connections. These connections bypass intermediate layers, effectively mitigating the vanishing gradient problem and allowing for the stable training of deep networks. ResNet-18 has a compact design, computationally efficient enough for training on consumer-grade hardware. Furthermore, it acts as a robust feature extractor due to

its early layers being already capable of detecting edges, curves, and basic shapes. The convolutional layers are frozen, leaving the pre-trained weights untouched.

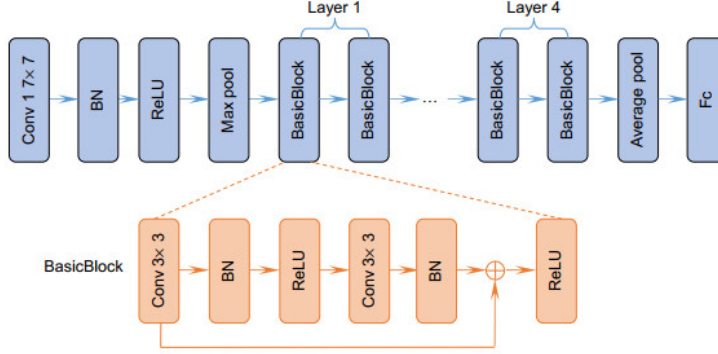


Figure 3: ResNet18 architecture

We reconstruct this model to fit our project goal by replacing the final fully connected layer with a newly initialized, untrained `nn.Linear` layer, outputting exactly 20 nodes, one for each predefined theme.

2.2 Loss Formulation

For our choice of loss function, we make the assumption that the themes corresponding to a certain Lego minifigure are independent of each other. That is, the presence of each theme is the outcome of an independent Bernoulli trial. To accommodate this, we use a Binary Cross-Entropy loss function combined with Logits (`BCEWithLogitsLoss`).

Unlike a Softmax function, which forces the outputs to sum to 1, this approach independently applies a Sigmoid activation function, $\sigma(\cdot)$, to every output node. For a single image, the multi-label loss $\mathcal{L}$ across C total classes (where $C = 20$) is defined mathematically as:

$$\mathcal{L} = - \sum_{c=1}^C [y_c \log(\sigma(\hat{y}_c)) + (1 - y_c) \log(1 - \sigma(\hat{y}_c))] \quad (1)$$

where $y_c \in \{0, 1\}$ is the ground-truth binary label for class c , and $\hat{y}_c$ is the raw logit prediction from the neural network. This allows the model to predict high probabilities for multiple themes simultaneously.

2.3 Handling Class Imbalance

In a real-world setting, themes follow a long-tail distribution; some themes (like Star Wars or City) have a significantly larger number of images compared to others. To handle this, we implement a two-pronged mathematical approach to force the model to respect minority classes.

First, we perform gradient starvation prevention by using `WeightedRandomSampler`. By using an inverse-frequency sampler with custom weights, we ensure that rare figures are oversampled and appear consistently in training.

Second, we constructed a custom `pos_weight` tensor, which was fed into our loss function. This heavily penalizes the optimizer if it incorrectly outputs a probability

approaching zero for a rare class, forcing it to aggressively update its weights when it ignores a minority theme.

2.4 Data Augmentation

We apply data augmentation by utilizing random horizontal flips and random rotation up to 15 degrees across our data splits to artificially expand the dataset and prevent the model from memorizing exact pixel layouts.

3 Experiments and Results

3.1 Evaluation Metrics

During training and validation, we employ our standard loss and accuracy as means of measuring model performance. However, for testing on our final data split, we choose to utilize F1-score instead, as standard confusion matrices are invalid for multilabel tasks, and using F1-score would give us a balance between Precision and Recall, which makes it the more mathematically rigorous choice.

3.2 Quantitative Results

We first discuss our training metrics, which were recorded during our 20-epoch run due to computational constraints. As can be seen from Figure 4, our training loss appeared to decrease steadily in a curve, before reaching a point where all improvements were marginal. In contrast, our validation loss appeared to reach a minimum during our 15th epoch before shooting up, indicating possible overfitting during our latter runs. Accuracy metrics showed the same trend, with the 15th epoch having the lowest difference between our training and validation accuracy, with possible signs of overfitting appearing as our epochs increased. We saved our best-performing model and used said model to evaluate on our test set. For this particular case, the best-performing model achieved an 80% accuracy with a loss of 0.038 on our validation set, decreasing by 20% from our initial validation loss.

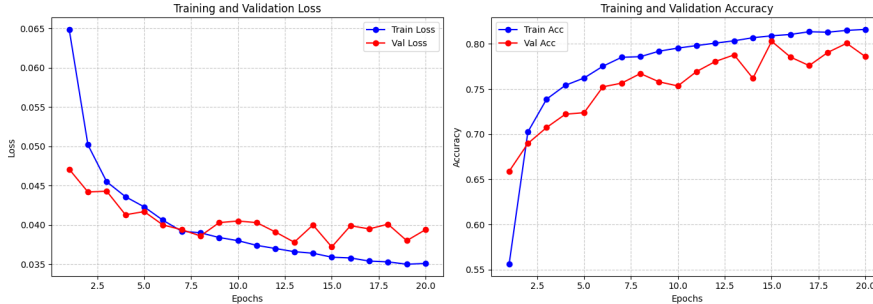


Figure 4: Training and validation loss

As illustrated in Figure 5, the model achieved varying levels of success across 20 themes. It performed exceptionally well on distinct themes such as Friends, Minecraft, Duplo and Star Wars. These are themes with highly unique structure differences, like

Minecraft’s blocky features or Friends’ unique hairpieces that the model identified efficiently. In contrast, it struggled with Castle, Icons and BrickLink, possibly due to sample scarcity and high visual overlap with other generic themes.

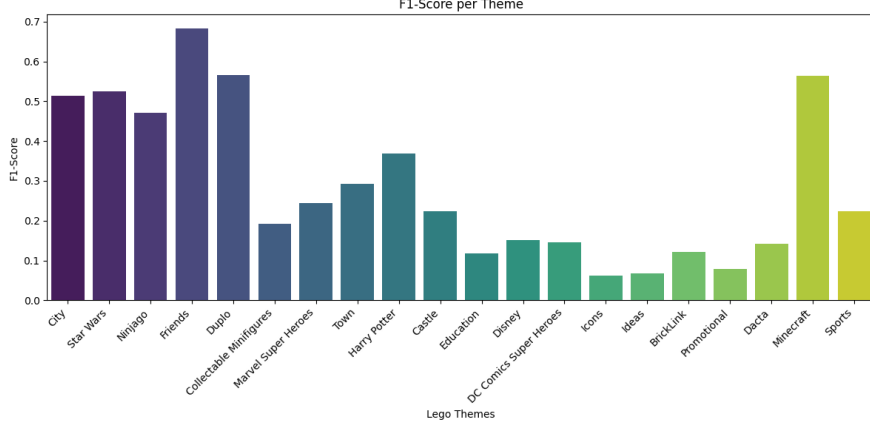


Figure 5: F1-scores on test set

4 Model Interpretability

To ensure the model is learning actual Lego features and to get a better understanding of the heuristics employed by our CNN, we implemented Gradient-Weighted Class Activation Mapping (Grad-CAM) [3], which visualizes the spatial gradients of the final convolutional layer to show exactly where the model focuses when making a prediction.

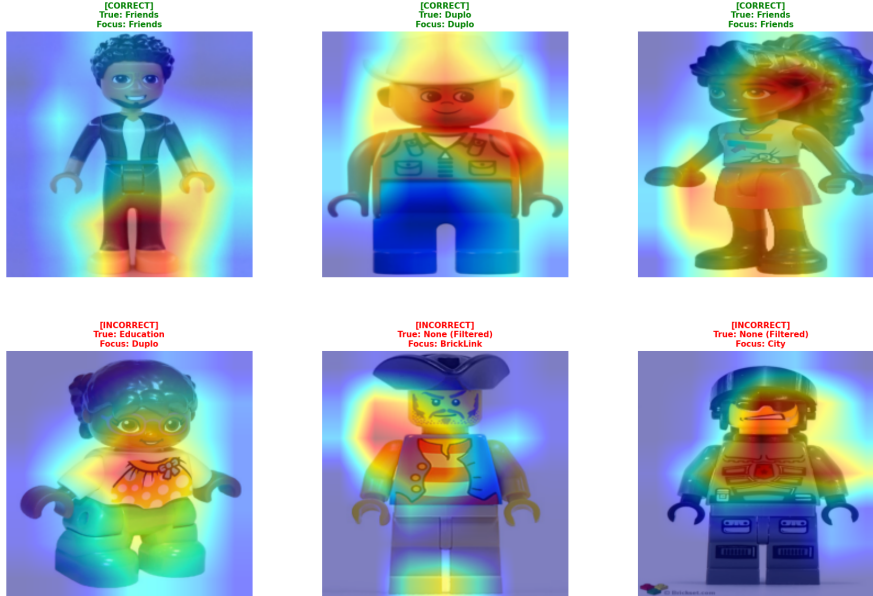


Figure 6: Grad-CAM interpretability result

From Figure 6, we can see that, for correct guesses, the model successfully learned discriminative, structural features. For the Friends figures, it consistently highlighted unique hairpieces and torso shapes. For the Duplo figures, it recognizes the oversized

facial proportions. The failures are also logical. In cases where the model incorrectly guessed Duplo for an Education figure, the Grad-CAM confirmed the model was looking at the oversized head and facial styling features which are visually identical to the Duplo themes. This proves that the model’s logic is visually sound, even when the ground-truth label differs.

5 Conclusion

In conclusion, by employing a transfer-learning architecture and different class-weighting techniques, we have successfully built a model capable of handling a multilabel problem. This sets the groundwork for future possible work, such as exploring more complex model architectures (e.g., Vision Transformers) to improve our performance, utilizing different techniques to deal with class imbalance, or diving deeper into different target objectives, like regression on manufacturing year.

References

- [1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).
- [2] Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., & Fei-Fei, L. (2009). *ImageNet: A large-scale hierarchical image database*. In 2009 IEEE conference on computer vision and pattern recognition (pp. 248-255). IEEE.
- [3] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). *Grad-CAM: Visual explanations from deep networks via gradient-based localization*. In Proceedings of the IEEE international conference on computer vision (pp. 618-626).
- [4] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). *PyTorch: An imperative style, high-performance deep learning library*. Advances in neural information processing systems, 32.
- [5] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). *Scikit-learn: Machine learning in Python*. Journal of machine learning research, 12(Oct), 2825-2830.
- [6] Brickset. (2024). *Brickset: The LEGO Minifigure Database*. Retrieved from <https://brickset.com/>